

ROBOTICS · LESSON 7

Driving by the Numbers

Count wheel ticks into real distance and command a setpoint — "drive 30 cm," not "drive for a second." The robot stops when it arrives, not when a timer expires.

ROBO-007 · ~40 MIN · INTERMEDIATE

1. The one idea

The robot knows how far it has gone. Count encoder ticks into real distance, and command a **setpoint** — "drive 30 cm" — instead of "drive for one second." The robot stops when it has *arrived*, not when a timer runs out.

2. Why it matters

Lesson 4 drove by time: `drive( ... ); delay(1000);`. That's a lie — the same delay covers a different distance on a fresh battery than a flat one, on carpet than tile. Counting how far the wheels actually turned (**odometry**) turns motion into a measurable distance. This is dead reckoning: the foundation of "go *there*," and the first thing under every robot that navigates.

3. Parts & wiring

The Lesson 6 setup, but **both** wheels have encoders now — left on GPIO18/19, right on GPIO32/33, each counted by its own interrupt into `ticksL` and `ticksR`. Everything else is the Lesson 4 rover.

The one new step is **calibration**: drive the robot a measured distance by hand or at a fixed speed, read the ticks, and divide. That gives you `TICKS_PER_CM` — the number that turns counts into centimetres for *your* wheels.

4. Predict (do this before uploading)

- If `TICKS_PER_CM` is 40 and you ask for 30 cm, how many ticks is the goal?
- You command 30 cm twice from the same spot. Will it land in the same place both times?
- Mid-drive, you nudge the robot so a wheel slips. Does it over-shoot or under-shoot the real distance?

5. Build it — command a goal, not a duration

```
// Found by calibration: drive a measured distance, read ticks, divide.
const float TICKS_PER_CM = 40.0;

// ticksL / ticksR are incremented by each wheel's encoder ISR (Lesson 6).
extern volatile long ticksL, ticksR;

void goDistance(float cm, int speed) {
    long startL = ticksL, startR = ticksR;
    long goal = (long)(cm * TICKS_PER_CM);
    drive(+1, +1, speed);           // Lesson 4 drive()
    while (true) {
        long dL = ticksL - startL;
        long dR = ticksR - startR;
        if ((dL + dR) / 2 ≥ goal) break;    // average wheel travel reached the
goal
        delay(5);
    }
    drive(0, 0, 0);                 // arrived - stop
}

void loop() {
    goDistance(30, 180);    // drive 30 cm and stop
    delay(2000);
}
```

The shape changed: you command a **goal**, the robot drives until its own odometry says it has reached it, then stops. It's closing a loop on *position* now, the way Lesson 6 closed one on speed. Pair it with the Lesson 6 PID per wheel and the two sides hold equal speed, so "straight" is actually straight.

6. Test against your prediction

Mark the floor, run `goDistance(30, 180)`, and measure. Within a centimetre or two? Run it again from the same mark — does it land in the same place? That repeatability is the point. Now push the robot mid-drive so a wheel slips: it overshoots, because odometry trusts the wheels, not the floor. That's the honest limit of dead reckoning.

7. What goes wrong (pre-loaded debugging)

- **Consistently over- or under-shoots** → `TICKS_PER_CM` is off. Re-calibrate over a longer run (more ticks = less error per cm).
- **Curves instead of going straight** → the wheels aren't matched. Add the Lesson 6 PID per wheel, or trim one side.
- **Drifts worse the farther it goes** → odometry error accumulates — wheel slip and surface matter. Dead reckoning is *relative*, never absolute; every serious robot eventually corrects it against something external (a wall, a beacon, a camera).
- **A turn lands at the wrong angle** → turning slips more than driving straight; calibrate ticks-per-degree separately, using the wheelbase.

8. Checkpoint

The robot drives by distance, not by time — it knows roughly how far it has gone and stops when it arrives. That's odometry, the seed of navigation. It's relative and it drifts, and seeing *that* clearly is what tells you why the next step is correcting it with something absolute.

9. Push further

- `goTurn(degrees)` : spin the wheels opposite and count ticks against the wheelbase.
- Chain moves into a path — drive a square (`goDistance` + `goTurn` ×4) and watch the drift pile up. A vivid demo of why pure dead reckoning isn't enough.
- Add an IMU later to correct heading drift on turns.